

MEMU - 1

E M U L A T E U R W 6 5 C 0 2

**** MEMO-1 ****

v24

CPU : W65C02S @ 0.75 MHz
RAM : 32 Ko (0000h - 7FFFh)
ROM : 16 Ko (C000h - FFFFh)
VIA : 65C22 (son + joysticks)
ACIA : 6551 (terminal Minitel 1B)
SON : Butterworth 2nd ordre x8
HDD : fichiers .bas (Ctrl+O / S)
CAS : cassette KCS WAV (SAVE/LOAD)

1-WOZMON 2-MS-BASIC 3-CASSETTE

MANUEL DE L'UTILISATEUR v24.9

Edition Claire -- Noir et Blanc

NOUVEAU v24 : SAVE/LOAD KCS -- cassette virtuelle WAV

BipCollector & Claude // BIP-SOFT 2026

Projet Memo-1 par Benoit Aveline (Memoire Morte)

Licence Creative Commons BY-NC-SA 4.0

[PRESS RESET TO BEGIN]

> QU'EST-CE QUE LE MEMO-1 ?

Le Memo-1 est un petit ordinateur a but pedagogique et ludique, conçu par Benoit Aveline (Memoire Morte). Il repose sur un processeur 65C02, la variante CMOS du legendaire MOS 6502 qui a equipe l'Apple II, le Commodore 64 ou encore la Nintendo NES.

L'originalite du Memo-1 est d'utiliser un Minitel 1B comme ecran et clavier. Tout passe par la liaison serie ACIA 6551, connectee au port peri-informatique DIN-5 du Minitel. Pas d'ecran integre, pas de clavier integre : le Minitel fait tout.

> L'EMULATEUR MEMU-1 v24

MEMU-1.EXE est un emulateur Windows du Memo-1 ecrit en C++17 :

- * CPU W65C02S cycle-exact (toutes instructions et modes d'adressage)
- * VIA 65C22 : Timer 1 auto-toggle pour le son, ports joystick Atari CX40
- * ACIA 6551 : liaison serie vers console Windows (remplace le Minitel)
- * RAM 32 Ko et ROM 16 Ko charges depuis memo1_rom.bin
- * ROM extension 8 Ko (cartouche) chargeable par glisser-deposer
- * Son Butterworth 2nd ordre + oversampling x8 + enveloppe ADSR
- * HDD\ : sauvegarde/chargement de programmes .bas (Ctrl+S / Ctrl+O)
- * CAS\ : cassette virtuelle KCS 300 baud WAV (SAVE / LOAD depuis BASIC)

> CONFIGURATION MATERIELLE EMULE

COMPOSANT	REFERENCE	DETAILS
CPU	W65C02S	0.75 MHz (25% sous-cadence volontaire)
RAM	HM62256B	32 Ko -- 0000h a 7FFFh
ROM	28C256	16 Ko -- C000h a FFFFh (memo1_rom.bin)
VIA	W65C22S	Timer 1 son + ports joystick + PB7 buzzer
ACIA	6551	Serie 19200 baud / 8N1 vers console Windows
SON	Buzzer	Butterworth x8 oversampling + ADSR 3ms/5ms
HDD	Virtuel	Dossier HDD\ -- fichiers .bas
CAS	Virtuel	Dossier CAS\ -- fichiers .wav KCS 300 baud
EXT	28C64	ROM extension 8 Ko -- A000h a BFFFh

> CARTE MEMOIRE COMPLETE

PLAGE	TAILLE	CONTENU
0000 - 00FF	256 o	Page zero : variables internes BASIC + 6502
0100 - 01FF	256 o	Pile 6502
0200 - 7FFF	~32 Ko	RAM utilisateur libre
8000 - 800F	16 o	VIA 65C22
9000 - 9003	4 o	ACIA 6551 (liaison serie)
A000 - BFFF	8 Ko	ROM cartouche extension J4
C000 - FFFF	16 Ko	ROM systeme (WOZMON + MS-BASIC)

> PAGE ZERO MS-BASIC (variables internes)

ADRESSE	NOM	ROLE
\$84/\$85	TXTTAB	Pointeur debut du programme BASIC en RAM
\$86/\$87	VARTAB	Pointeur debut des variables (= fin du prog)
\$88/\$89	ARYTAB	Pointeur debut des tableaux
\$8C/\$8D	FRETOP	Pointeur haut de la zone des chaines
\$7A/\$7B	FAC1	Accumulateur flottant 1 (calculs)
\$F5/\$F6	ADDR	Adresse KCS (utilisee par la ROM apres LOAD)
\$F7/\$F8	LENGTH	Longueur KCS (utilisee par la ROM apres LOAD)
\$0200/\$0201	PC	Adresse affichee apres LOADED

> REGISTRES VIA 65C22 (8000h)

ADRESSE	NOM	ROLE
8000h	PortB	Port B -- bit 7 (PB7) = sortie buzzer son
8002h	DDRB	Direction port B (0x80 = PB7 en sortie)
8004h	T1CL	Timer 1 compteur octet bas (ecriture = latch)
8005h	T1CH	Timer 1 haut + charge latch + demarre le timer
800Bh	ACR	Bits 7-6=11 : auto-toggle PB7 (mode TONE actif)

> CONTENU DU ZIP

FICHIER / DOSSIER	DESCRIPTION
MEMU1.exe	Executable principal (autonome, pas d'installation)
memo1_rom.bin	ROM systeme 32 Ko -- INDISPENSABLE dans le meme dossier
compile.bat	Script de recompilation (necessite MinGW-w64 / GCC 13)
SRC\	Code source complet C++17 avec en-tetes BIP-SOFT
HDD\	Dossier disque dur virtuel -- fichiers .bas
CAS\	Dossier cassette virtuelle -- fichiers .wav KCS

> PREMIER LANCEMENT

- * Extraire le ZIP dans un dossier de votre choix
- * Verifier la presence de memo1_rom.bin dans le meme dossier que MEMU1.exe
- * Double-cliquer sur MEMU1.exe -- une console Windows s'ouvre
- * La melodie de demarrage confirme que le son fonctionne

[CONSOLE AU DEMARRAGE]

```
[MEMO-1] Son active (buzzer PB7).
[MEMO-1] Ctrl+S=sauver  Ctrl+O=charger  Ctrl+Q=quitter
[MEMO-1] SAVE/LOAD BASIC -> cassette KCS dans CAS\

*** MEMO-1 ***

1 - WOZMON   (moniteur memoire Steve Wozniak, Apple 1 1976)
2 - MS-BASIC (interpreteur Microsoft 1975)
3 - CASSETTE (menu sauvegarde KCS cassette physique)
4 - ROM EXT  (si cartouche detectee a A000h)
A - A PROPOS
R - RESTART BASIC
```

> RACCOURCIS CLAVIER

TOUCHE	ACTION
Ctrl + Q	Quitter l'emulateur proprement (arret audio inclus)
Ctrl + S	Sauvegarder le programme BASIC courant dans HDD\
Ctrl + O	Charger un fichier .bas depuis HDD\ (menu interactif)

> PRINCIPE

L'émulateur fournit un "disque dur" virtuel via deux raccourcis clavier.
Ce système est indépendant du BASIC : il capture la sortie de LIST
pour SAVE, et reinjecte le texte ligne par ligne pour LOAD.
Les fichiers .bas sont de simples fichiers texte lisibles dans Notepad.

> SAUVEGARDER (Ctrl+S)

[SAUVEGARDE HDD]

[Appuyer Ctrl+S]
Nom du fichier (sans extension) : monprog
[SAUVEGARDE] monprog.bas -> HDD\monprog.bas (342 octets)

> CHARGER (Ctrl+O)

[CHARGEMENT HDD]

```
+-----+
| CTRL+O -- CHARGEMENT FICHIER BASIC |
+-----+
| 1. exemple.bas |
| 2. monprog.bas |
| 3. test_tone.bas |
+-----+
| Numero ou nom (Entree = annuler) : 2
+-----+
[OK] 8 lignes injectees depuis HDD\monprog.bas
```

! Ne pas appuyer sur des touches pendant le chargement.
Pour un fichier de 30 lignes, comptez environ 15 secondes.
Taper LIST après le chargement pour vérifier le programme.

> QU'EST-CE QUE LE KCS ?

Le Kansas City Standard (KCS) est un protocole de sauvegarde audio sur cassette, standardisé en 1976. Le Memo-1 l'utilise à 300 baud. L'émulateur génère et lit des fichiers WAV à la place d'une cassette.

> ENCODAGE KCS 300 BAUD

BIT	FREQUENCE	CYCLES	DETAIL
Bit 0	1200 Hz	4 cycles	8 demi-périodes par bit
Bit 1	2400 Hz	8 cycles	16 demi-périodes par bit

Trame UART par octet :

```
[start=0][D0 D1 D2 D3 D4 D5 D6 D7 LSB first][stop=1][stop=1]
= 11 bits au total par octet
```

[STRUCTURE BLOC KCS MEMO-1]

Structure d'un bloc KCS Memo-1 :

```
[Silence 0.5s]
[Leader : 1500 x bit-1 à 2400Hz -- ~5s de synchro]
[Magic : 0x4D "M" 0x31 "1"]
[Adresse de chargement : 2 octets little-endian]
[Longueur des données : 2 octets little-endian]
[Données : N octets]
[Checksum : 1 octet = XOR de tous les octets data]
[Silence 0.5s]
```

> SAVE ET LOAD DEPUIS BASIC

Depuis MS-BASIC, taper simplement :

```
SAVE -> génère CAS\SAVE_XXXX_date_heure.wav
LOAD -> affiche la liste CAS\*.wav, choisir par numéro
```

> SAUVEGARDER AVEC SAVE

[SAUVEGARDE KCS]

```
> SAVE
Press REC then any key          ; message ROM intercepte
[KCS-SAVE] 342 octets sauvegardes ($0200-$0357)
[KCS-SAVE] WAV -> CAS\SAVE_0200_20260729_143022.wav
SAVED
```

> CHARGER AVEC LOAD

[CHARGEMENT KCS]

```
> LOAD
+-----+
| KCS LOAD -- CHARGEMENT CASSETTE |
+-----+
| 1. SAVE_0200_20260729_143022.wav |
+-----+
| Numero ou nom (Entree = annuler) : 1
|
[KCS-LOAD] Decodage de SAVE_0200_20260729_143022.wav ...
[KCS-LOAD] 342 octets charges a $0200 -- overlay ROM actif
Loaded.
$0200
```

> COMPATIBILITE AVEC LES SCRIPTS PYTHON

Les WAV generes par SAVE sont compatibles avec memo12bin.py.
Les WAV generes par bin2memo1.py sont chargeable via LOAD.
Les WAV fonctionnent aussi sur la vraie machine Memo-1.

! Taper NEW avant LOAD pour vider la memoire.
Taper LIST apres LOADED pour verifier le programme.
RUN pour executer.

> PRESENTATION

WOZMON est le moniteur memoire de l'Apple 1 : 256 octets de code pur.
 Il permet d'examiner et modifier la memoire octet par octet, et
 d'executer du code machine directement. Toutes les valeurs en HEX.
 Taper \ pour revenir au menu de demarrage.

> COMMANDES

SYNTAXE	EFFET
AAAA	Affiche le contenu de l'octet a l'adresse AAAA
AAAA.BBBB	Dump hexadecimal de la plage AAAA a BBBB inclus
AAAA: XX YY ...	Ecrit les octets XX YY... a partir de AAAA
AAAA R	Execute le code machine a partir de AAAA
\	Retour au menu de demarrage ROM

> EXEMPLE : LIRE ET MODIFIER LA MEMOIRE

[LECTURE / ECRITURE]

```
\ 0300
0300: 00      ; contenu actuel
\ 0300: A9
0300: 00      ; WOZMON reaffiche l'ANCIEN contenu (normal)
\ 0300
0300: A9      ; nouveau contenu confirme
\ 0300.030F   ; dump de 16 octets
0300: A9 00 00 00 00 00 00 00
0308: 00 00 00 00 00 00 00 00
```

> EXEMPLE : ECRIRE ET EXECUTER UN PROGRAMME

[MINI-PROGRAMME LANGAGE MACHINE]

```
; LDA #$41 = A9 41 (charger "A")
; STA $9000 = 8D 00 90 (ecrire dans ACIA TX)
; RTS = 60
\ 0200: A9 41 8D 00 90 60
\ 0200.0205   ; verification
0200: A9 41 8D 00 90 60
\ 0200 R      ; execution
A             ; "A" affiche par le programme
```


> LIRE LES VECTEURS 6502

[VECTEURS 6502]

```

\ FFFA.FFFF
FFFA: 00 0F      ; NMI   -> 0F00h
FFFC: DA E1      ; RESET -> E1DAh (little-endian)
FFFE: FB F3      ; IRQ   -> F3FBh

```

> ACCEDER AU VIA DEPUIS WOZMON

[ACCES DIRECT AU VIA]

```

; Mettre PB7 en sortie
\ 8002: 80
; PB7 = HIGH -> buzzer ON
\ 8000: 80
; PB7 = LOW -> buzzer OFF
\ 8000: 00
; Lire les joysticks via PortA
\ 8001
8001: 1F          ; 0x1F = aucun bouton presse

```

> LIRE LES VARIABLES BASIC DEPUIS WOZMON

[VARIABLES BASIC EN PAGE ZERO]

```

; Lire TXTTAB (debut programme BASIC)
\ 0084.0085
0084: 00 02      ; TXTTAB = $0200

; Lire VARTAB (fin programme = debut variables)
\ 0086.0087
0086: 57 03      ; VARTAB = $0357

; Longueur programme = VARTAB - TXTTAB = $0357 - $0200 = 343 octets

```

! Les adresses 0000-00FF (page zero) sont utilisees par le BASIC.
 Zone sure pour vos programmes manuels : 0200h - 7FFFh.

> HISTORIQUE

L'interpreteur MS-BASIC embarque est la lignee directe du premier produit commercial de Microsoft (1975) pour l'Altair 8800. Port 6502 par Ric Weiland. Desassemblage et restauration par Michael Steil (pagetable.com). Chaque instruction commence par un NUMERO DE LIGNE (10, 20, 30...).

> COMMANDES DE BASE

COMMANDE	EFFET
LIST	Affiche le programme en memoire
LIST 10-50	Affiche uniquement les lignes 10 a 50
RUN	Execute depuis la premiere ligne
NEW	Efface le programme ET les variables
CONT	Reprend apres STOP
PRINT expr	Affiche valeur ou texte. ; = colle, , = tabule
INPUT "msg";v	Affiche msg et attend une saisie
GET a\$	Lit un caractere sans attendre Entree
LET v = expr	Affecte une valeur (LET optionnel)
REM texte	Commentaire, ignore a l'execution
END / STOP	Termine / interrompt le programme
SAVE	Sauvegarde sur cassette KCS (WAV dans CAS\)
LOAD	Charge depuis cassette KCS (menu CAS\)
POKE adr,val	Ecrit un octet directement en memoire
PEEK(adr)	Lit un octet directement en memoire
TONE val	Joue une note (0=silence, specifique Memo-1)
JOY(n)	Lit l'etat du joystick 0 ou 1 (specifique Memo-1)

07. MS-BASIC (2/3) -- STRUCTURES ET FONCTIONS

> STRUCTURES DE CONTROLE

STRUCTURE	DESCRIPTION
FOR i=1 TO n	Boucle avec compteur
FOR i=1 TO n STEP s	Boucle avec pas s
NEXT i	Fin de boucle FOR
IF cond THEN instr	Execution conditionnelle
GOTO n	Saut inconditionnel
GOSUB n / RETURN	Appel de sous-programme / retour
ON x GOTO a,b,c	Saut selon valeur de x
WHILE cond / WEND	Boucle tant que cond vraie

> FONCTIONS DISPONIBLES

FONCTION	DESCRIPTION
ABS(x)	Valeur absolue
INT(x)	Partie entiere (arrondi vers le bas)
RND(x)	Nombre aleatoire entre 0 et 1
SQR(x)	Racine carree
LOG(x) / EXP(x)	Logarithme naturel / exponentielle
SIN(x)/COS(x)/TAN(x)	Trigonometrie (radians)
ASC(a\$)	Code ASCII du premier caractere
CHR\$(n)	Caractere correspondant au code ASCII n
LEN(a\$)	Longueur d'une chaine
LEFT\$(a\$,n) / RIGHT\$(a\$,n)	Extraction debut / fin de chaine
MID\$(a\$,p,n)	Extraction depuis position p
STR\$(x) / VAL(a\$)	Conversion nombre <-> chaine
PEEK(n)	Lit l'octet en memoire a l'adresse n
FRE(0)	Memoire libre restante en octets
JOY(n)	Etat joystick n (Memo-1)

> EXEMPLE 1 : CALCULS ET SAISIE

[CALCULS]

```

10 INPUT "ENTRER UN NOMBRE : "; N
20 IF N <= 0 THEN PRINT "POSITIF SVP" : GOTO 10
30 PRINT "RACINE  = "; SQR(N)
40 PRINT "DOUBLE  = "; N * 2
50 PRINT "CARRE   = "; N ^ 2
60 PRINT "REFAIRE ? (O/N)"
70 GET A$ : IF A$ = "O" THEN GOTO 10
80 END

```

> EXEMPLE 2 : TABLEAUX ET SOUS-PROGRAMMES

[TABLEAUX]

```

10 DIM A(10)
20 FOR I=1 TO 10 : A(I)=INT(RND(1)*100) : NEXT I
30 GOSUB 200 : GOSUB 300 : END
200 FOR I=1 TO 10 : PRINT A(I);" " : NEXT I
210 PRINT "" : RETURN
300 S=0 : FOR I=1 TO 10 : S=S+A(I) : NEXT I
310 PRINT "SOMME=";S;" MOYS=";S/10 : RETURN

```

> EXEMPLE 3 : CYCLE SAVE / LOAD KCS

[SAVE / LOAD KCS]

```

10 PRINT "BONJOUR MEMO-1 v24 !"
20 FOR I=1 TO 5 : PRINT I : NEXT I
30 END

SAVE          ; -> CAS\SAVE_xxxx_date.wav

NEW           ; effacer la memoire
LOAD          ; -> menu CAS\ -> choisir -> Loaded.
LIST          ; verifier
RUN           ; executer

```

> EXEMPLE 4 : JOYSTICK ET SON

[JOYSTICK + SON]

```

10 A = JOY(0)
20 IF A=30 THEN PRINT "HAUT" : TONE 1277
30 IF A=29 THEN PRINT "BAS" : TONE 957
40 IF A=15 THEN PRINT "FEU !" : TONE 852
50 FOR I=0 TO 60 : NEXT I : TONE 0
60 GOTO 10

```

> CHAINE DE TRAITEMENT AUDIO v24

ETAPE DE TRAITEMENT	
[1]	OVERSAMPLING x8 : 8 sous-echantillons par sample audio
[2]	FILTRE BUTTERWORTH anti-aliasing fc=6kHz sur flux 352kHz
[3]	DECIMATION 8->1 : moyenne des 8 sous-echantillons filtres
[4]	FILTRE BUTTERWORTH final fc=3500Hz sur flux 44100Hz
[5]	ENVELOPPE ADSR : attack 3ms (132 samples) / release 5ms (220)
[6]	GAIN + CLAMP int16 : amplitude 28000, sortie waveOut 44100Hz

> FORMULE ET TABLE DES NOTES (0.75 MHz)

[FORMULE TONE]
$\text{valeur_TONE} = 750000 / (\text{frequence} * 2)$ Exemple LA 440 Hz : $750000 / 880 = 852$

NOTE	FREQUENCE	VALEUR	EXEMPLE BASIC
DO	261.63 Hz	1433	TONE 1433 ; Do central C4
DO#	277.18 Hz	1353	TONE 1353
RE	293.66 Hz	1277	TONE 1277
RE#	311.13 Hz	1205	TONE 1205
MI	329.63 Hz	1138	TONE 1138
FA	349.23 Hz	1074	TONE 1074
FA#	369.99 Hz	1014	TONE 1014
SOL	392.00 Hz	957	TONE 957
SOL#	415.30 Hz	903	TONE 903
LA	440.00 Hz	852	TONE 852 ; Diapason standard
LA#	466.16 Hz	804	TONE 804
SI	493.88 Hz	759	TONE 759

> **bin2memo1.py -- BINAIRE VERS WAV**

[bin2memo1.py]

```
python3 bin2memo1.py --bin code.bin --start 0x6C00 --out tape.wav
```

```
Input:    code.bin
Start:    $6C00
Length:   1024 bytes
Output:   tape.wav (6.2 s)
Done.
```

> **memo12bin.py -- WAV VERS BINAIRE**

[memo12bin.py]

```
python3 memo12bin.py tape.wav --out code.bin
```

```
Input:    tape.wav
Format:   44100 Hz, 274155 samples (6.2 s)
Start:    $6C00
Length:   1024 bytes
Output:   code_6C00.bin
Done.
```

! Les WAV generes par SAVE dans l'emulateur sont lisibles par memo12bin.py.

Les WAV generes par bin2memo1.py sont chargeable via LOAD.

Compatibilite totale avec la vraie machine Memo-1.

> PROBLEMES GENERAUX

SYMPTOME	SOLUTION
Ecran noir / pas de menu	Verifier memo1_rom.bin dans le meme dossier
Pas de melodie au demarrage	Verifier peripherique audio Windows (waveOut)
Craquements continus	Bug latch T1LH -- utiliser v22+ ou recompiler
CTRL+0 / saisie decallee	Bug echo console -- utiliser v24 corrige
Lignes superposees au LOAD	Corrige v23 : injection ligne par ligne
ROM extension ignoree	ROM non programmee ou connecteur inverse

> PROBLEMES KCS

SYMPTOME	SOLUTION
SAVE : pas de fichier WAV	Verifier dossier CAS\ present a cote de MEMU1.exe
SAVE : pas de message KCS	Verifier raw_buf -- utiliser v24
LOAD : menu vide	Aucun .wav dans CAS\ -- y placer des fichiers
LOAD ERROR	Checksum invalide ou WAV corrompu
Curseur bloque apres LOAD	Bug ancien -- corrige v24 overlay ROM

> HISTORIQUE DES VERSIONS

VER.	MODIFICATION
v21	Premier son : Timer 1 auto-toggle + AudioEngine waveOut 44100Hz
v22	Fix latch T1LH : write(T1CH) met a jour regs[7] et regs[6]
v23	CPU 0.75 MHz, Fix Ctrl+0, AudioEngine Butterworth + ADSR
v24	SAVE/LOAD KCS WAV, raw_buf ACIA, ROM overlay E389=CLC+RTS

> MATERIEL ORIGINAL

Benoit Aveline (Memoire Morte)
<https://github.com/MemoireMorte/Memo-1>
Licence Creative Commons BY-NC-SA 4.0

> EMULATEUR -- BIP-SOFT 2026

Conception et integration : Bipcollector
Code C++17 : Claude (Anthropic)
Contributions IA : Kimi, Lovable, Gemini, ChatGPT

> LOGICIELS EN ROM

WOZMON : Steve Wozniak (Apple 1, 1976)
MS-BASIC : Gates, Allen, Weiland (Microsoft, 1975)
Port 6502 : Ric Weiland
Desassemblage : Michael Steil (pagetable.com)
ca65 : assembleur 6502 -- <https://cc65.github.io>

> MUSIQUES D'AMBIANCE

Berlinadine
Forever Damned - Victorian Gothic Punk Rock
Death to the Machines! - Geek Rock Alt Rock Punk
Hyakki Kodan (Fuma-kai Magatsurai-za)

> REFERENCES TECHNIQUES

Ben Eater -- "Build a 6502 computer from scratch" (CC-BY)
6502.org -- Opcodes, datasheets W65C02S / 6522 / 6551
WDC -- <https://www.westerndesigncenter.com>
BYTE Magazine, Novembre 1976 -- Kansas City Standard

*** FIN DU MANUEL ***

BIP-SOFT -- Bastion Interplanetaire Positronique -- 2026

> POURQUOI UN FICHIER WAV ?

Le Memo-1 original sauvegarde sur cassette audio via un module KCS.
 Le signal encode les bits en deux frequences : 1200 Hz pour 0, 2400 Hz pour 1. L'emulateur reproduit ce mecanisme numeriquement : SAVE genere un fichier WAV mono 44100 Hz / 16 bits identique a une vraie cassette.
 Ce WAV peut etre joue sur la vraie machine -- et elle le lit correctement.

> STRUCTURE D'UN BLOC KCS MEMO-1

CHAMP	DESCRIPTION
Silence 0.5s	Samples audio a zero avant le signal
Leader ~5s	1500 bits-1 a 2400 Hz -- synchro du decodeur
Magic 4D 31h	Signature "M1" identifiant le Memo-1
Adresse depart	2 octets little-endian (lo, hi)
Longueur	2 octets little-endian (lo, hi)
Donnees N octets	Le programme BASIC brut depuis TXTTAB
Checksum XOR	1 octet = XOR de tous les octets de donnees
Silence 0.5s	Samples audio a zero apres le signal

> ENCODAGE BIT PAR BIT

Chaque bit est encode en onde carree, les deux ont la MEME duree (3.33ms):
 bit-1 (2400 Hz) : 8 cycles complets = 16 demi-periodes de 156 cycles
 bit-0 (1200 Hz) : 4 cycles complets = 8 demi-periodes de 312 cycles

Trame UART par octet (11 bits) :

[start=0][D0 D1 D2 D3 D4 D5 D6 D7 LSB first][stop=1][stop=1]

[ENCODAGE KCS DE 0x4D "M"]

```
; 0x4D = 0100 1101 en binaire, LSB first : 1,0,1,1,0,0,1,0
[start=0][1][0][1][1][0][0][1][0][stop][stop]
1200Hz  2400 1200 2400 2400 1200 1200 2400 1200 2400  2400
```

> COMMENT LA ROM DECODE LE SIGNAL KCS

La ROM lit le port materiel \$B001 (carte KCS connectee en \$A000-\$BFFF).

Cinq routines collaborent pour decoder un octet complet :

ROUTINE	ROLE
E313	Detecte UNE transition de \$B001. Compte X lectures identiques. $X < 5 = \text{bruit}$, $5 \leq X < 19 = \text{bit-1}$, $X \geq 19 = \text{bit-0}$.
E349	Lit \$B001 pendant 163 cycles, compte les transitions. $>12 = \text{bit-1}$ (2400Hz), $\leq 12 = \text{bit-0}$ (1200Hz). Utilise ROR.
E369	Lit UN octet : 2x bit-0 (start), 6 transitions (E30B), 8 bits via E349 assembles par ROR.
E33A	Sync leader : appelle E313 en boucle, attend 200 appels consecutifs bit-1 (signal 2400Hz continu = 1).
E389	Routine principale : E33A + E369 x N. Verifie checksum. Carry CLEAR = OK, carry SET = LOAD ERROR.

> POURQUOI E313 FONCTIONNE

E313 boucle en lisant \$B001 toutes les ~16 cycles CPU :

$\text{INX}(2) + \text{BEQ}(2) + \text{LDA } \$B001(4) + \text{AND}(2) + \text{CMP}(3) + \text{BEQ}(3) = 16 \text{ cycles/iteration}$

A 0.75 MHz, une demi-periode KCS dure :

bit-1 (2400 Hz) : $750000 / (2 \times 2400) = 156 \text{ cycles} \rightarrow X \sim 9 \text{ iterations}$

bit-0 (1200 Hz) : $750000 / (2 \times 1200) = 312 \text{ cycles} \rightarrow X \sim 19 \text{ iterations}$

Seuil CPX #\$13 (19) : $X \geq 19 = \text{bit-0}$, $X < 19 = \text{bit-1}$.

Marge entre 9 et 19 : tolere bien les variations d'une vraie cassette.

[E313 DESASSEMBLE]

```

E313: LDA $B001 : AND #$01 : STA $F4 : LDX #$00
LOOP: INX
      LDA $B001 : AND #$01 : CMP $F4
      BEQ LOOP      ; identique -> continuer a compter
      CPX #$05      ; X < 5 -> bruit, ignorer
      BCS NEXT
      STA $F4 : BRA LOOP
NEXT: CPX #$13      ; X >= 19 -> bit-0
      BCS BIT0
      SEC : RTS      ; bit-1 (carry SET)
BIT0: CLC : RTS      ; bit-0 (carry CLEAR)

```

> LE DEFI

La ROM s'attend a lire un signal analogique depuis une vraie cassette.
Dans l'emulateur, il n'y a ni cassette ni port KCS materiel.
Le defi : controler ce que Bus::read(\$B001) retourne pour que la ROM
croie lire une vraie cassette.

> 6 TENTATIVES -- DE L'ECHEC A LA SOLUTION

TENTATIVE	RESULTAT
1. Reads individuels	N copies identiques par demi-periode. ECHEC : E313, E2E8 et E349 lisent a des rythmes d
2. READS_LONG=21	Ajustement pour X=20>=19. ECHEC partiel : premier octet OK mais ?LOAD ERROR persiste. D
3. Transitions en cycles	kcs_cycle += 4 (LDA=4 cycles). ECHEC : les 10 autres cycles entre deux LDA (INX, AND, C
4. tickKCS par cycle	tickKCS() appele a chaque cpu.clock(). ECHEC : leader de 200 bit-1 KCS = 37 secondes re
5. Leader reduit + trailer	reader de 13 bit-1 seulement. ECHEC : simulation cycle-accurate de E313/E349 trop diffi
6. ROM overlay (SOLUTION)	Bus::read(E389)=CLC+RTS. Donnees ecrites directement en RAM. Variables ZP mises a jour.

> LA SOLUTION : ROM OVERLAY

[ROM OVERLAY EN ACTION]

```
; ROM inchangee a F952 :
F952: JSR $E389    ; appel routine KCS receive
F955: BCS $F9BD    ; si carry SET -> LOAD ERROR
F957: ...          ; suite -> "Loaded."

; Bus::read() avec overlay actif :
E389 -> $18 (CLC) ; efface le carry
E38A -> $60 (RTS) ; retour immediat a F955

; carry=0 -> BCS non pris -> "Loaded." s'affiche
; Donnees deja ecrites en RAM avant l'appel
; $F5/$F6 = start_addr, $F7/$F8 = length, $0200/$0201 = addr
```

> NOUVEAUTES v24

TYPE	DESCRIPTION
KCS	SAVE : encode programme BASIC en WAV KCS 300 baud dans CAS\
KCS	LOAD : decode WAV, overlay ROM E389=CLC+RTS, Loaded. affiche
KCS	WAV compatible avec vraie machine et scripts Python
CODE	raw_buf ACIA : texte brut sans ANSI pour detection KCS
CODE	ROM overlay Bus::read(E389/E38A) transparent pour la ROM
CODE	En-tete BIP-SOFT dans tous les fichiers sources .h et .cpp
BUG	Fix detection : getDisplayBuf() -> getRawBuf() (sans codes ANSI)
BUG	Fix readByte KCS.h : pos par reference (bool) au lieu de size_t
BUG	Fix curseur bloqué : ROM overlay contourne E389 hardware

> EN-TETE BIP-SOFT (PRESENT DANS TOUS LES FICHIERS SOURCES)

```
// =====
// MEMU-1 - Emulateur Memo1
// =====
// Par Bipcollector, Claude, Kimi
// Lovable, Gemini & ChatGPT
// -----
// Musiques :
//   Berlinadine
//   Forever Damned - Victorian Gothic Punk Rock
//   Death to the Machines! - Geek Rock Alt Rock Punk
//   Hyakki Kodan - Fuma-kai Magatsurai-za
// -----
// Une Creation BIP-SOFT - 2026
// =====
// Bastion Interplanetaire Positronique
// Bastion numerique dediee a la creation humaine
// par des systemes artificiels
// =====
```

*** FIN DU MANUEL ***

BIP-SOFT -- Bastion Interplanetaire Positronique -- 2026